

IA Y ML EN AZURE · JORNADA AZURE ML

Azure Machine Learning Studio

Del experimento al modelo operable

4 horas · master class · caso [Aurora Energía](#)

Qué nos llevaremos hoy

- Entender **qué es Azure ML Studio** y dónde encaja.
- Preparar **datos, compute y entornos**.
- Ejecutar experimentos con **notebooks, jobs y MLflow**.
- Revisar **AutoML, Designer y pipelines**.
- Registrar modelos y desplegar **endpoints online o batch**.
- Conectar Azure ML con **Foundry y Fabric**.

Hilo conductor: priorización y predicción de incidencias para **Aurora Energía**.

Agenda · 4 h

BLOQUE	MIN	TEMA
M0	15	Mapa Azure AI/ML y caso
M1	30	Qué es Azure ML Studio
M2	35	Datos, compute y entornos
M3	35	Experimentos, notebooks y MLflow
☕	15	Descanso
M4	35	AutoML, Designer y pipelines
M5	35	Registro, endpoints e inferencia
M6	30	MLOps, gobierno y monitorización
M7	20	Integración con Foundry y Fabric
M8	10	Cierre



M1 · 30 min

Azure ML Studio

La superficie de trabajo de Azure Machine Learning

Azure ML Studio en una frase

Azure Machine Learning Studio es el portal para **preparar datos, entrenar, comparar, registrar, desplegar y operar** modelos de machine learning en Azure.

PIEZA	PARA QUÉ SIRVE
Workspace	Contenedor de activos y ejecuciones
Data	Datasets, datastores y versiones
Compute	Notebooks, jobs, clusters y endpoints
Jobs	Ejecuciones trazables
Models	Registro y versionado
Endpoints	Inferencia online y batch
MLflow	Tracking portable

Azure ML vs Foundry vs Fabric

NECESIDAD	PLATAFORMA
Agentes, tools, RAG, evaluación generativa	Foundry
Entrenar modelos predictivos propios	Azure ML
Registrar y desplegar modelos ML	Azure ML
Lakehouse, BI y pipelines analíticos	Fabric
Predicciones dentro de un agente	Foundry + endpoint Azure ML
Predicciones en informes	Azure ML + Fabric

No son productos excluyentes: se reparten responsabilidades.

Caso Aurora

Aurora quiere priorizar incidencias operativas:

- Fabric contiene históricos y reporting.
- Azure ML entrena un modelo de riesgo/prioridad.
- Foundry usa la predicción como contexto o tool.
- Operaciones revisa métricas y casos dudosos.



```
Datos -> modelo predictivo -> predicción -> agente / dashboard
```

Ideas clave

- **Workspace** = unidad operativa de Azure ML.
- **Job** = ejecución trazable.
- **Modelo registrado** = candidato reutilizable.
- **Endpoint** = consumo gobernado.
- **Fabric y Foundry** amplían el flujo, no sustituyen Azure ML.

M2 · 35 min

Datos, compute y entornos

La base antes de entrenar

Datos en Azure ML

ELEMENTO	USO
Datastore	Conexión a almacenamiento
Data asset	Dataset versionado
MLTable	Lectura tabular declarativa
OneLake / Fabric	Origen o destino analítico

Evitar rutas locales frágiles es el primer paso hacia MLOps.

Compute

COMPUTE	ENCAJE
Compute instance	Notebook, exploración, debugging
Compute cluster	Entrenamiento, AutoML, pipelines
Serverless	Jobs puntuales si está disponible
Online endpoint	Inferencia síncrona
Batch endpoint	Scoring masivo

Un cluster con mínimo 0 evita coste residual.

Entornos reproducibles

Un environment fija:

- Imagen base.
- Paquetes Conda/pip.
- Dependencias del entrenamiento.
- Dependencias de inferencia.
- Requisitos de sistema.

Si no puedes repetir el entorno, no puedes confiar en el resultado.

Demo · Preparar workspace

1. Abrir `https://ml.azure.com` .
2. Entrar en `mlw-aurora-curso` .
3. Revisar Storage y datastore por defecto.
4. Registrar `aurora-incidencias` .
5. Revisar `ci-aurora-*` y `cpu-cluster` .
6. Mostrar un environment gestionado.

M3 · 35 min

Experimentos y MLflow

Trazabilidad desde el primer notebook

Qué registra un buen experimento

SEÑAL	EJEMPLO
Parámetros	Algoritmo, profundidad, features
Métricas	F1, RMSE, MAE, latencia
Artefactos	Gráficas, matriz de confusión
Modelo	Carpeta MLflow o formato propio
Código/entorno	Versión ejecutada

Sin tracking, el modelo no es repetible ni auditable.

MLflow mínimo

```
import mlflow

with mlflow.start_run():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)

    mlflow.log_param("max_depth", 6)
    mlflow.log_metric("f1_weighted", f1)
    mlflow.sklearn.log_model(model, "model")
```

Comparar ejecuciones

RUN	MODELO	PARÁMETRO	F1	DECISIÓN
001	RandomForest	depth 4	0.72	Baseline
002	RandomForest	depth 6	0.78	Candidato
003	LogisticRegression	C 1.0	0.69	Descartar

La mejor métrica no siempre gana si el modelo es lento, frágil o difícil de explicar.

Demo · Entrenar con tracking

1. Abrir notebook.
2. Cargar incidencias.
3. Preparar features sencillas.
4. Entrenar modelo base.
5. Registrar métricas con MLflow.
6. Cambiar parámetro y comparar runs.

M4 · 35 min

AutoML, Designer y pipelines

Tres formas de avanzar hacia producción

Opciones de trabajo

OPCIÓN	ENCAJE
Notebooks	Exploración y prototipo
AutoML	Baseline rápido
Designer	Flujo visual y formación
Pipelines	Repetición y automatización
CLI/SDK	CI/CD y control

AutoML

AutoML ayuda a explorar:

- Algoritmos.
- Transformaciones.
- Hiperparámetros.
- Métricas comparables.
- Límites de tiempo y coste.

AutoML acelera el baseline; no reemplaza el criterio técnico.

Designer

Designer permite construir flujos visuales:

- Ingesta.
- Transformación.
- Entrenamiento.
- Evaluación.
- Registro.

Útil para explicar el proceso y alinear equipos mixtos.

Pipelines



```
Preparar datos  
-> entrenar  
-> evaluar  
-> registrar si cumple  
-> desplegar o puntuar
```

Pipeline es el puente entre notebook y operación.

Demo · Opciones en Studio

1. Mostrar AutoML y sus decisiones mínimas.
2. Mostrar Designer como grafo visual.
3. Revisar un pipeline como flujo repetible.
4. Conectar con Fabric como origen de datos.
5. Explicar cómo llevarlo a CLI/SDK.

M5 · 35 min

Modelos y endpoints

Entrenar no es entregar

Registro de modelos

Registrar un modelo permite:

- Versionar artefactos.
- Separar experimento de candidato.
- Asociar métricas y linaje.
- Promocionar entre entornos.
- Reutilizar en endpoints y batch.

Online vs batch

TIPO	USO
Online endpoint	Respuesta síncrona para apps, APIs o agentes
Batch endpoint	Scoring masivo o programado sobre ficheros

PREGUNTA	PISTA
Necesito respuesta inmediata	Online
Tengo millones de filas nocturnas	Batch
Quiero escribir predicciones en lakehouse	Batch
Foundry llama a una tool predictiva	Online

Piezas de un deployment

PIEZA	FUNCIÓN
Modelo	Artefacto registrado
Environment	Dependencias
Scoring script	<code>init()</code> y <code>run()</code>
Deployment	Configuración de compute
Endpoint	URL, auth, logs y tráfico

Demo · Consumo

1. Registrar `aurora-prioridad-incidencia`.
2. Crear o revisar endpoint online.
3. Probar payload pequeño.
4. Revisar logs.
5. Comparar con batch scoring.
6. Apagar o eliminar recursos de prueba.

M6 · 30 min

MLOps y gobierno

El modelo como servicio mantenible

Ciclo mínimo

1. Versionar datos, código y configuración.
2. Ejecutar entrenamiento como job o pipeline.
3. Registrar métricas y artefactos.
4. Validar criterios de promoción.
5. Registrar modelo.
6. Desplegar endpoint.
7. Monitorizar y reentrenar.

Controles

ÁREA	CONTROL
Identidad	Entra ID, RBAC, managed identity
Red	Private endpoints si hay datos sensibles
Datos	Permisos mínimos y linaje
Coste	Clusters a cero, cuotas, apagado
Calidad	Métricas, drift, regresión
Auditoría	MLflow, logs, Azure Monitor

Riesgos habituales

- Notebook que funciona solo en la máquina de una persona.
- Dataset sin versión.
- Modelo registrado sin métrica de aceptación.
- Endpoint olvidado consumiendo coste.
- Drift no monitorizado.
- Permisos amplios sobre datos sensibles.

M7 • 20 min

Foundry + Fabric

Arquitectura conjunta

Patrón de referencia



Fabric / ADLS Gen2

- > preparación de datos y features
- > Azure ML job / pipeline
- > modelo registrado
- > endpoint online o batch
- > aplicación, informe o agente Foundry

Decisión por responsabilidad

RESPONSABILIDAD	PLATAFORMA
Lakehouse y reporting	Fabric
Entrenamiento predictivo	Azure ML
Registro y endpoint ML	Azure ML
Agente conversacional	Foundry
Tool predictiva para agente	Foundry llama a Azure ML
Predicciones en BI	Azure ML escribe, Fabric consume

Cierre

- Azure ML Studio cubre el ciclo de vida de modelos ML.
- Foundry convierte capacidades en aplicaciones y agentes.
- Fabric aporta datos, lakehouse, pipelines analíticos y reporting.
- La arquitectura correcta separa responsabilidades y une las plataformas con identidad, permisos y trazabilidad.

La pregunta no es "qué portal usamos", sino "qué responsabilidad tiene cada plataforma".